lab_anomaly 学习流程任务总结

本文档围绕 lab_anomaly 项目，从核心概念、数据流、模型、训练、推理五大维度，构建“从看懂到能改”的学习路径，以下为关键要点梳理：

一、三大核心基础概念

是理解后续流程的前提，定义项目中核心数据单元与核心任务逻辑：

- **Clip**：模型最小输入单位，指一段短视频片段（如16帧，每隔2帧取1帧）；一条视频会均匀采样多个 Clip。
- **Embedding**：Clip 经 ViT（视觉Transformer）编码后生成的固定维度向量（如768维），后续分类、开放集检测均基于此向量，不直接使用像素数据。
- **开放集**：训练仅用“正常”样本，推理时可识别“未见过”的异常；本项目通过 **KMeans + One-Class SVM** 在正常样本的 Embedding 上建模实现。

二、阶段一：数据流——从 CSV 到 Clip 再到磁盘 Embedding

目标

明确“一条视频如何转化为若干 Clip，再生成向量并存储到磁盘”的完整链路。

关键任务与核心内容

任务	核心文件	需理解的关键逻辑
1.1 标注与索引	`data/video_labels.py`、`data/index_build.py`	1. CSV 每行对应一条视频，`label` 仅支持 `normal`/具体异常类名/`unknown`；`start_time`/`end_time` 可选（用于截取视频片段） 2. 从 `raw_videos/` 扫描视频，生成/更新 `video_labels.csv`，新视频默认 label 为 `unknown`（需人工修改）
1.2 视频→Clip 采样	`data/clip_dataset.py`、（可选）`data/video_reader.py`	1. 关键参数：`clip_len`（Clip 长度）、`frame_stride`（帧步长）、

		`num_clips_per_video`（每视频采样 Clip 数量） 2. 采样逻辑：按参数将视频切分为 N 个 Clip（每个 Clip 是“若干帧的列表”），若有 `start_time` / `end_time` 则仅在该区间采样
1.3 Clip→Embedding 落盘	`train/extract_embeddings.py`、`configs/embedding_example.yaml`	1. 存盘格式区别：`npy_per_clip`（每个 Clip 一个 npy 文件）、`npz_per_video`（每个视频一个 npz 文件） 2. `embeddings_meta.jsonl`：每行对应一个 Clip/视频，含 `embedding_path` / `label` / `video_id` 等信息，后续训练依赖此文件

阶段检验

能绘制流程图，清晰呈现“CSV → 按行读视频 → 每视频采 N 个 Clip → 每个 Clip 输入 ViT → 生成 Embedding 存盘 + 写入 meta 文件”的全流程（可手画或用 mermaid）。

三、阶段二：模型——ViT 编码器 + MIL 分类头

目标

明确“Clip 如何生成向量、多 Clip 如何转化为视频级预测”，并掌握修改模型结构的关键位置。

关键任务与核心内容

任务	核心文件	需理解的关键逻辑
2.1 Clip→单向量（Encoder）	`models/vit_video_encoder.py`	1. 输入输出：输入为“帧列表的列表”或归一化的 `(B,T,C,H,W)` 张量，输出为 `(B,D)` 维度的 Embedding（B=批量大小，D=向量维度） 2. 核心流程：`processor` 预处理 → HuggingFace VideoMAE 模型编码 → `pooling`

		（auto/pooler/cls/mean）选择最终向量表示 3. 扩展点：换 backbone 仅需修改 `model_name`；改输入分辨率/帧数需调整 `processor` 与配置
2.2 多 Clip→视频级预测（MIL 头）	`models/mil_head.py`	1. 输入输出：输入为 `(B,N,D)` 张量（N=Clip 数量），输出为视频级 logits `(B,C)`（C=类别数） 2. 聚合逻辑：`AttentionPooling`（学习权重加权聚合）、`topk`（取分数最高的 k 个 Clip 平均） 3. 扩展点：修改 `pooling` 方式、`topk` 数值等需调整 `MILHeadConfig`；新增 pooling 方式需在 `MILClassifier` 中加分支

阶段检验

能口头解释：“一个视频的 8 个 Clip → 生成 8 个 768 维向量 → MIL 头聚合为 1 个向量 → 线性层输出 C 类 logits”的完整过程。

四、阶段三：训练——三种核心产物（Embedding / 已知分类器 / 开放集模型）

目标

掌握三个训练脚本的“输入-输出-关键参数”，明确修改参数/逻辑的位置。

关键任务与核心内容

任务	核心文件	需理解的关键逻辑
3.1 提取 Embedding（回顾）	`train/extract_embeddings.py`、`configs/embedding_example.yaml`	可修改参数：`clip_len` / `frame_stride` / `num_clips_per_video` / `model_name` / `batch_size` / `save_format`
3.2 已知异常分类器		

	`train/train_known_classifier.py`	1. 输入：已缓存的 Embedding 路径（非视频）；`--exclude_unknown` 参数可过滤 label=unknown 的样本 2. 输出：`checkpoint_best.pt`（含 `model_state` / `label2idx` / `idx2label` / `cfg`） 3. 可修改参数：`--pooling` / `--epochs` /学习率/优化器；修改类别集合仅需更新 CSV 的 label 并重新训练
3.3 开放集模型（KMeans + OCSVM）	`train/fit_kmeans_ocsvm.py`	1. 核心流程：仅取 label=normal 的 Embedding → KMeans 聚类 → 每簇（或全局）训练 One-Class SVM → 计算 `anomaly_score = -decision_function` → 按分位数确定阈值 2. 推理逻辑：先通过 KMeans 判断 Clip 所属簇，再用对应 OCSVM 和阈值判断是否异常（小簇回退到全局 OCSVM） 3. 可修改参数：`--k`（KMeans 聚类数）/ `--min_cluster_size` / `--nu`（OCSVM 参数）/ `--gamma` / `--quantile`（阈值分位数）

阶段检验

能回答：1. 已知分类器使用哪些 label？2. 开放集模型仅用哪种 label？3. 分类器 checkpoint 包含哪些内容？4. `thresholds.json` 有哪些 key？

五、阶段四：推理——打分融合与 RTSP 实时服务

目标

明确一次推理的完整链路，掌握修改配置/新增输出（如 API、告警）的关键位置。

关键任务与核心内容

任务	核心文件	需理解的关键逻辑
4.1 开放集打分 + 已知分类 + 融合	`infer/scoring.py`	1. 开放集打分：KMeans 判簇 → 选对应 OCSVM → 计算 `anomaly_score` → 与阈值比较（`score>阈值` 则判异常） 2. 已知分类：置信度低于 `min_known_prob` 则视为 `unknown` 3. 融合结果：含 `predicted_label` / `predicted_prob` / `anomaly_score` / `is_anomaly`
4.2 RTSP 实时流水线	`infer/rtsp_service.py`、`configs/rtsp_service_example.yaml`	1. 核心流程：按 `sample_fps` 抽帧 → 滑窗组成 Clip → 调用编码器 → 已知分类+开放集打分 → 融合 → 去抖（`min_consecutive` / `cooldown_sec`）→ 写 `events.jsonl` /截图/可选 HTTP POST 2. 配置分工：`artifacts`（产物路径）、`encoder`（编码器配置）、`sampling`（采样参数）、`fusion`（融合阈值）、`outputs`（输出方式）、`runtime`（运行参数） 3. 可修改点：RTSP 地址、checkpoint/开放集模型路径、抽帧/滑窗参数、融合阈值、API URL；新增告警条件需在“融合后-写事件前”加分支

阶段检验

1. 能绘制“RTSP 帧 → 滑窗 Clip → 编码器 → 已知分类+开放集打分 → 融合 → 去抖 → 输出”的流程图；2. 能指出修改 YAML 中哪些参数会影响推理敏感度或延迟。

六、扩展操作速查（“想做的事”对应“核心文件/配置”）

想做的操作	核心涉及文件/配置
修改 Clip 长度、帧步长、每视频 Clip 数	`configs/embedding_example.yaml`、`configs/rtsp_service_example.yaml`、各脚本的 `--clip_len` / `--frame_stride` / `--num_clips_per_video` 参数
更换视频编码模型（如其他 HuggingFace 模型）	`models/vit_video_encoder.py`（修改 `model_name`）、Embedding/RTSP 配置中的 `encoder.model_name`
修改已知分类器的 pooling 方式/结构	`models/mil_head.py`（`MILHeadConfig`）、`train/train_known_classifier.py`
调整开放集模型的敏感度/误报率	`train/fit_kmeans_ocsvm.py` 的 `--k` / `--quantile` / `--nu` 参数（阈值已写入 `thresholds.json`，推理端无需额外参数）
修改融合策略（何时判为 unknown/异常）	`infer/scoring.py`（`fuse_known_and_open_set` 函数）、`configs/rtsp_service_example.yaml`（`fusion.*` 配置）
新增输出方式（如其他 API、数据库）	`infer/rtsp_service.py`（在“写 `events.jsonl` / 截图/POST”逻辑附近加代码）

七、建议学习顺序与时间

1. **阶段一（1~2小时）**：按“1.1标注与索引→1.2视频→Clip采样→1.3 Clip→Embedding落盘”读代码，必要时在 CSV/ `embeddings_meta.jsonl` 加打印/断点观察数据流转；

2. **阶段二（1小时）**：理解 ViT 编码器与 MIL 头的输入输出维度、配置逻辑；可写测试代码加载模型，喂假数据观察输出形状；
3. **阶段三（1~2小时）**：先用少量数据跑通“提取 Embedding→训练已知分类器→训练开放集模型”流程，再回头梳理脚本逻辑；
4. **阶段四（1小时）**：理清 `scoring.py` 与 `rtsp_service.py` 的调用关系；有 RTSP 资源可跑实时服务，修改 YAML 观察行为变化。

每完成一个阶段，通过该阶段“检验”自测；全部阶段完成后，尝试修改1~2处参数/配置，确认能定位到对应代码位置，即达到“能改”目标。

（注：文档部分内容可能由 AI 生成）